

Software Systems

Assignment 3

System Context: The SkyHigh Booking System

SkyHigh is a rapidly expanding airline company. As their business has grown, their core software system has become increasingly difficult to maintain. The development team has identified four major pain points in the current codebase.

Please read through the four problem scenarios below and complete the tasks outlined in the "Your Mission" section.

The Scenarios

Problem 1: The Pain of Adding New Aircraft

SkyHigh's fleet is growing. We have the usual `PassengerPlane`, some `Helicopters` for special tours, and we just bought a fleet of futuristic `PassengerDrones`. The problem is, adding a new aircraft type to the system is a nightmare. The code that creates flights has giant `if-else` blocks that check the aircraft model. To add an "Airship," a developer has to find every single one of these blocks and add another `else if`. It's a slow, error-prone process that everyone on the team hates.

Problem 2: The "Monster Constructor" for Orders

When a customer makes a reservation, we create an `Order` object. This object holds everything: passenger details, flight info, price, and optional extras like travel insurance, extra baggage, or meal choices. To handle all this, the `Order` class has one huge constructor with a dozen parameters. Most of them are optional, so creating a simple order looks like this: `new Order(customer, flights, null, 2, null, ...)`. It's confusing, easy to get the order of parameters wrong, and a constant source of bugs.

Problem 3: The Integration Nightmare

Our aircraft come from different manufacturers, and their internal software systems are all different. This means our `ScheduledFlight` class has to get information (like passenger capacity) in different ways:

- The `PassengerPlane` class has a public variable: `public int passengerCapacity;`
- The `Helicopter` class has a function: `public int getPassengerCapacity();`
- The `PassengerDrone` doesn't have a function for this at all. The code just has to "know" its capacity is 4.

To handle this, the code is filled with ugly `instanceof` checks. As discussed in the lectures, this is a classic design flaw. It breaks the Open/Closed Principle because our main code has to change every time we get a new type of aircraft.

Problem 4: The All-Powerful Payment Class

The **FlightOrder** class is responsible for everything related to a booking, including processing payments. It has one giant method with a long **if-else** chain to handle different payment types like "CreditCard" or "PayPal." Now, the business team wants to partner with new payment providers like "CryptoWallet" and "BankTransfer." The developers are worried because adding more **else-if** statements will make the **FlightOrder** class even bigger and harder to test. It's becoming a "God Class" that tries to do too much.

Your Mission

From the four problems described above, your task is to:

1. **Select three** major design flaws that you believe can be fixed with design patterns.
2. **Propose a solution** for each of the three flaws using a suitable design pattern.
3. **Constraint:** Across your total of three solutions, you must use **at least one Creational, one Structural, and one Behavioral** pattern.

Deliverables & Formatting

For **each** of the three patterns you choose, your submission must include the following sections:

1. **The Problem:** Briefly state the problem you are solving in your own words.
2. **Chosen Design Pattern and Category:** Name the pattern and its specific category (Creational, Structural, or Behavioral).
3. **Justification:** Explain *why* this pattern is the right tool for the job. How does it make the system more flexible or easier to maintain? Connect your explanation to the design principles we've discussed in class, such as low coupling, high cohesion, the Open/Closed Principle, or separation of concerns.
4. **Solution Sketch:** Show your proposed idea with a simple UML diagram (Class or Sequence) OR some pseudo-code. The goal is to clearly illustrate the *structure* of your solution, not to write perfect, compilable code.
5. **Assumptions (If any):** Real-world problem descriptions are never perfect. If you feel a detail is missing to implement your pattern, it is okay to make a reasonable assumption. Just be sure to state your assumptions clearly here.

Submission Guidelines:

- Submit your answers as a single PDF document.
- Ensure any diagrams are clear, readable, and properly labeled.